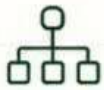


System Design Concepts

1. Load Balancer



Use Cases:

- Web applications
- APIs
- Microservices

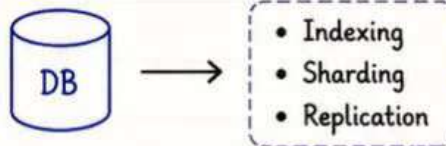
2. Cache



Use Cases:

- Session store
- API responses
- Leaderboards

3. Database



Use Cases:

- User data
- Transactions
- Analytics

4. Microservices



Use Cases:

- E-commerce apps
- Banking systems
- SaaS platforms

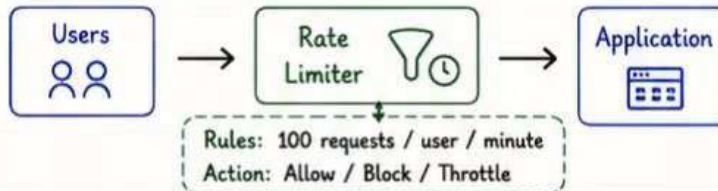
5. Message Queue



Use Cases:

- Order processing
- Email sending
- Event-driven systems

6. Rate Limiter



Use Cases:

- APIs
- Login attempts
- Payment requests

7. Search Service



Use Cases:

- E-commerce search
- Document search
- Log search

8. File Storage



Use Cases:

- Media files
- User uploads
- Static assets
- Backup & Archive

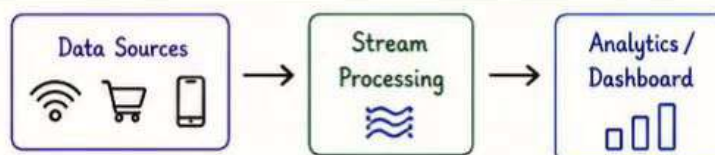
9. Notification System



Use Cases:

- Alerts
- Reminders
- Updates
- Promotions

10. Real-time Analytics



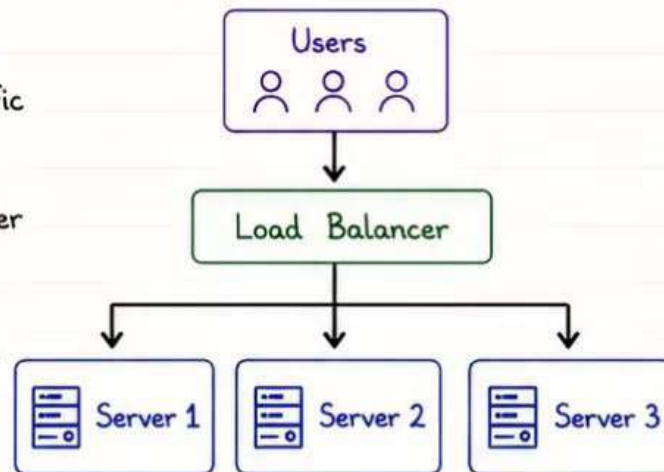
Use Cases:

- Dashboards
- Fraud detection
- Live monitoring
- User behavior analytics

System Design Concepts

1. Load Balancer

- Distributes incoming traffic across multiple servers.
- Prevents any single server from being **overloaded**.
- Improves **availability** and **fault tolerance**.



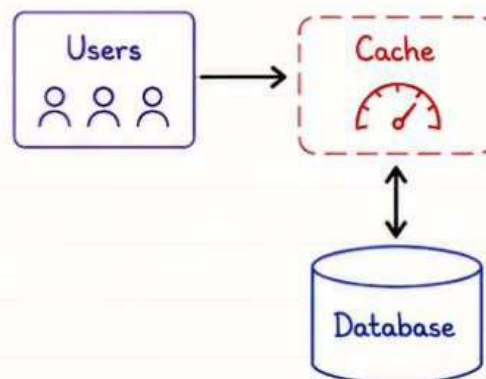
Use Cases:

- Web applications
- APIs
- Microservices

★ It acts as the single entry point and ensures traffic is routed efficiently to healthy servers.

2. Cache

- Stores frequently accessed data in memory.
- Reduces database load and improves **response time**.
- Data in cache is temporary but **very fast**.



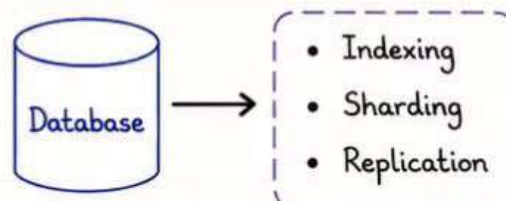
Use Cases:

- Session store
- API responses
- Leaderboards
- Product catalog

★ Cache sits between users and database to serve data quickly and reduce latency.

3. Database

- Stores, manages and organizes data **permanently**.
- We can choose type based on needs (SQL / NoSQL).
- **Important concepts:** Indexing, Sharding, Replication.



Use Cases:

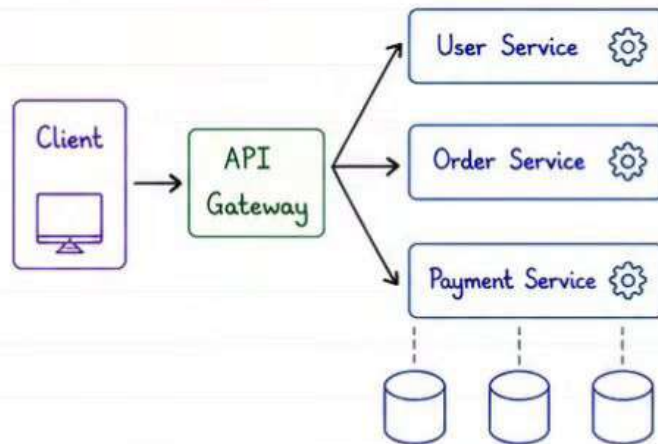
- User data
- Transactions
- Analytics

★ Database is the backbone for persistent storage and ensures data consistency and reliability.

System Design Concepts

4. Microservices

- Breaks a large application into small, independent services.
- Each service has its own responsibility and database.
- Improves scalability, flexibility and deployment.



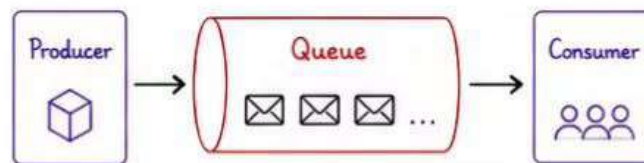
Use Cases:

- E-commerce apps
- Banking systems
- SaaS platforms

★ Microservices allow teams to develop, deploy and scale services independently, resulting in a more agile system.

5. Message Queue

- Enables asynchronous communication between services.
- Messages are stored in a queue until consumed.
- Improves decoupling and reliability of the system.



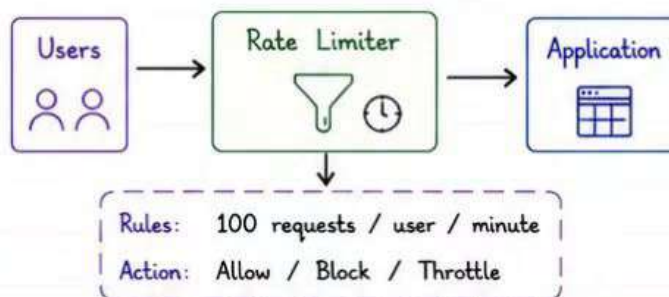
Use Cases:

- Order processing
- Email sending
- Event-driven systems

★ Message queues act as a buffer between services and ensure data is not lost even if a service is temporarily down.

6. Rate Limiter

- Limits the number of requests a user can make in a time window.
- Prevents abuse and protects system from overload.
- Ensures fair usage of resources.



Use Cases:

- APIs
- Login attempts
- Payment requests

★ Rate limiting helps maintain system stability and provides a better experience for all users.

System Design Concepts

7. Search Service

- Provides fast and relevant search results.
- Uses indexing and text analysis.
- Handles millions of search queries efficiently.



Use Cases:

- E-commerce search
- Document search
- Log search

★ Search services help users find what they need quickly using relevant results.

8. File Storage

- Stores and serves large files like images, videos, documents.
- Offloads files from app servers.
- Improves performance and scalability.



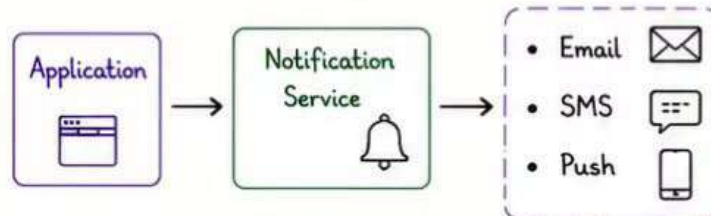
Use Cases:

- Media files
- User uploads
- Static assets
- Backup & Archive

★ File storage services provide durable, scalable and cost-effective storage for all types of files.

9. Notification System

- Sends real-time notifications through multiple channels.
- Decouples notification logic from main application.
- Ensures timely delivery and better user engagement.



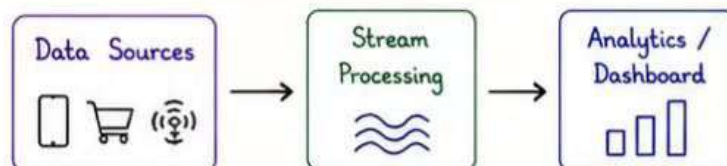
Use Cases:

- Alerts
- Reminders
- Updates
- Promotions

★ Notification systems keep users informed and engaged through the right channel at the right time.

10. Real-time Analytics

- Processes data in real-time and generates insights.
- Works on streaming data.
- Helps in decision making and monitoring.



Use Cases:

- Dashboards
- Fraud detection
- Live monitoring
- User behavior analytics

★ Real-time analytics turns live data into actionable insights for faster and smarter decisions.



These building blocks together help us design scalable, reliable and efficient systems!